

Chapter 4

List

列表

Gaoyang Shan

Ajou University

目录

1. 单向链表和链表---Singly Linked Lists and Chains
2. 在C语言中表示链表---Representing Chains in C
3. 链式栈和链式队列---Linked Stacks and Queues
4. 多项式---Polynomials
5. 列表的附加操作---Additional List Operations
6. 双向链表---Doubly Linked Lists

单向链表和链表

❖ 顺序表示

- 列表中连续的元素[items]以固定的距离间隔存储

❖ 链的表示

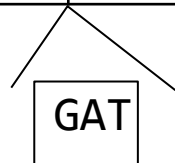
- 元素可以存储在内存中的任何位置
- 为了访问列表中的元素
 - 存储该列表中下一个元素的地址或位置

为什么使用链表

(BAT, CAT, EAT, FAT, HAT, JAT, LAT, MAT, OAT, PAT, RAT, SAT, VAT, WAT) 排序的有序列表

如果使用顺序映射,

BAT	CAT	EAT	FAT	HAT	JAT	LAT	MAT	OAT	PAT	RAT	SAT	VAT	WAT	
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--



怎样插入 GAT?

BAT	CAT	EAT	FAT	GAT	HAT	JAT	LAT	MAT	OAT	PAT	RAT	SAT	VAT	WAT
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

怎样删除 LAT?

BAT	CAT	EAT	FAT	GAT	HAT	JAT	MAT	OAT	PAT	RAT	SAT	VAT	WAT	
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	--

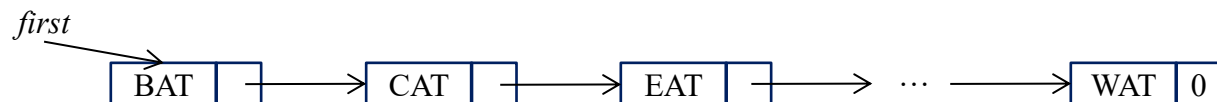
插入和删除任意元素的成本变得很高!

单向链表和链表

❖ $data[i]$ 和 $link[i]$ 共同构成一个节点

- 数据 $[data]$: 元素不再按顺序存储
- 链接 $[Link]$: 这些值是指向数据数组中元素的指针
- 列表检索开始于 $data[8]=BAT$
 - $first=8$
 - $link[8]=3$, 这意味着指向 $data[3]$, 其为 CAT
- 当我们到达有序列表的末尾时
 - $Link$ 等于0

	<i>data</i>	<i>link</i>
1	HAT	15
2		
3	CAT	4
4	EAT	9
5		
6		
7	WAT	0
8	BAT	3
9	FAT	1
10		
11	VAT	7

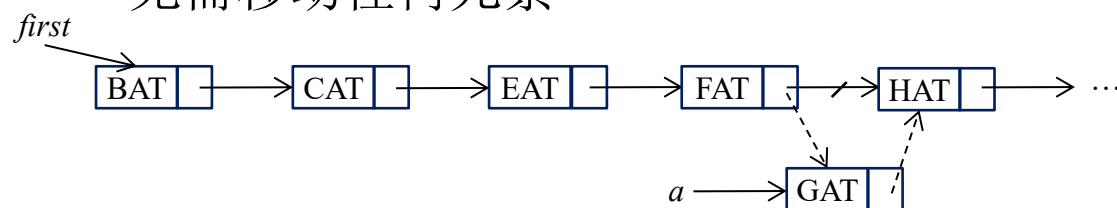


绘制链接列表的常规方法

单向链表和链表

❖ 插入

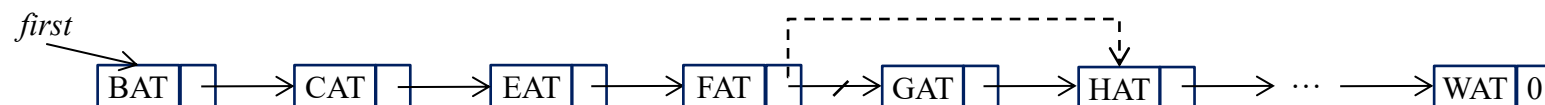
- 插入 GAT 到列表中
 - 无需移动任何元素



	<i>data</i>	<i>link</i>
1	HAT	15
2		
3	CAT	4
4	EAT	9
5	GAT	1
6		
7	WAT	0
8	BAT	3
9	FAT	5
10		
11	VAT	7

❖ 删除

- 从列表中删除 GAT
 - 即使 GAT 的链接 [link=1] 仍然指向 HAT，但由于 GAT 不再位于链表的有效路径中，即无法从链表的头节点遍历到 GAT，因此它不再属于链表的一部分。



C语言中的链表[chain]

- ❖ 链表[chain]是一种链式结构[linked list]，其中每个节点[node]表示一个元素
- ❖ 从一个元素到下一个元素之间有一个链接或指针

```
typedef struct listNode *listPointer;  
typedef struct listNode {  
    char data[4];  
    listPointer link; /* 自引用结构*/  
};
```

C语言中的链式结构

❖ 示例 [*List of words*]

- 创建一个空列表

- `listPointer first = NULL;`

/*声明一个指向链表头节点的指针first, 并将其初始化为NULL, 表示当前链表为空*/

- 用于测试链表是否为空的宏 [Macro]

- `#define IS_EMPTY(first) (!first)`

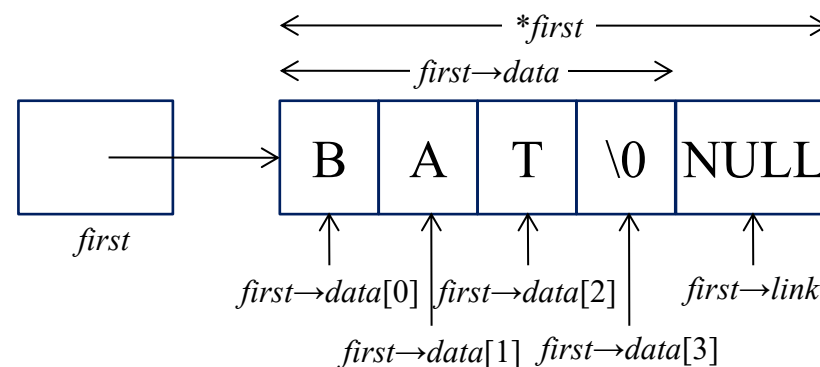
- 创建一个新节点

- `MALLOC( first, sizeof(*first) );`

- 将词组 BAT 放入列表 [list]

- `strcpy( first→data, "BAT" );`

- `first→link = NULL;`



→ : 结构体成员运算符

`first → data = (*first).data`

C语言中的链式结构

❖ 示例：两节点链表 [*Two-node linked list*]

```
typedef struct listNode *listPointer;
typedef struct listNode {
    int      data;
    listPointer link;    /* 自引用结构*/
};
```

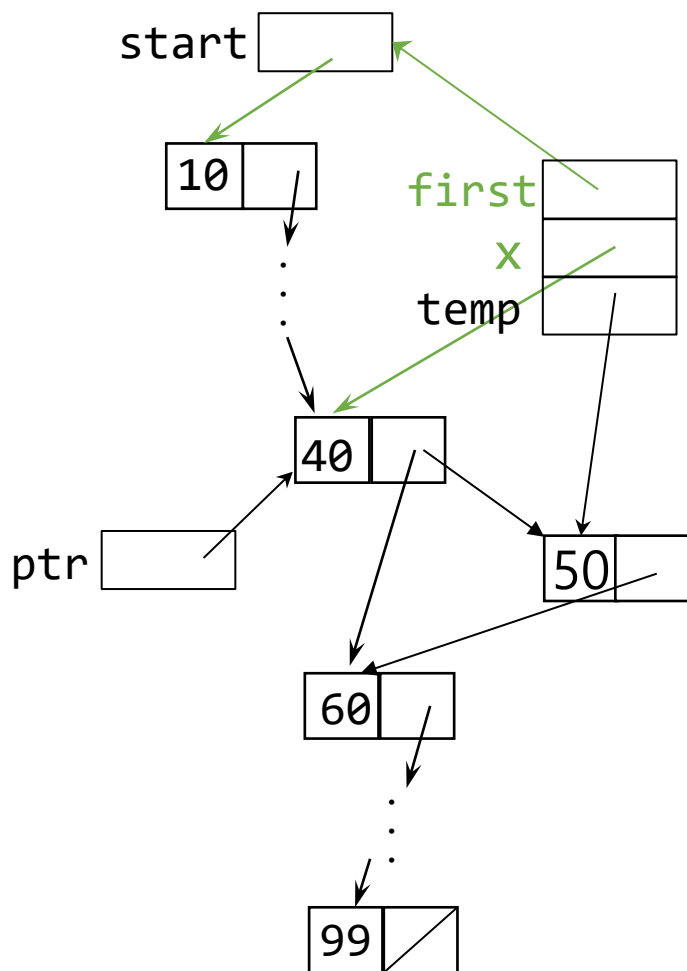
```
listPointer create2()
{
    /*创建有两个节点的链表*/
    listPointer first, second;
    MALLOC( first, sizeof(*first) );
    MALLOC( second, sizeof(*second) );
    second->link = NULL;
    second->data = 20;
    first->data=10;
    first->link = second;
    return first;
}
```



列表插入

```
void insert(listPointer* first, listPointer x)
{ /*在节点 x 之后插入一个 data=50 的一个新节点到链表 [chain] first 中*/
    listPointer temp;
    MALLOC(temp, sizeof(*temp));
    temp->data = 50;
    if(*first) { /* if (*first != NULL), 如果链表[chain]是非空的*/
        temp->link = x->link;
        x->link = temp;
    }
    else { /* if (*first == NULL), 如果链表[chain]是空的*/
        temp->link = NULL;
        *first = temp;
    }
}
```

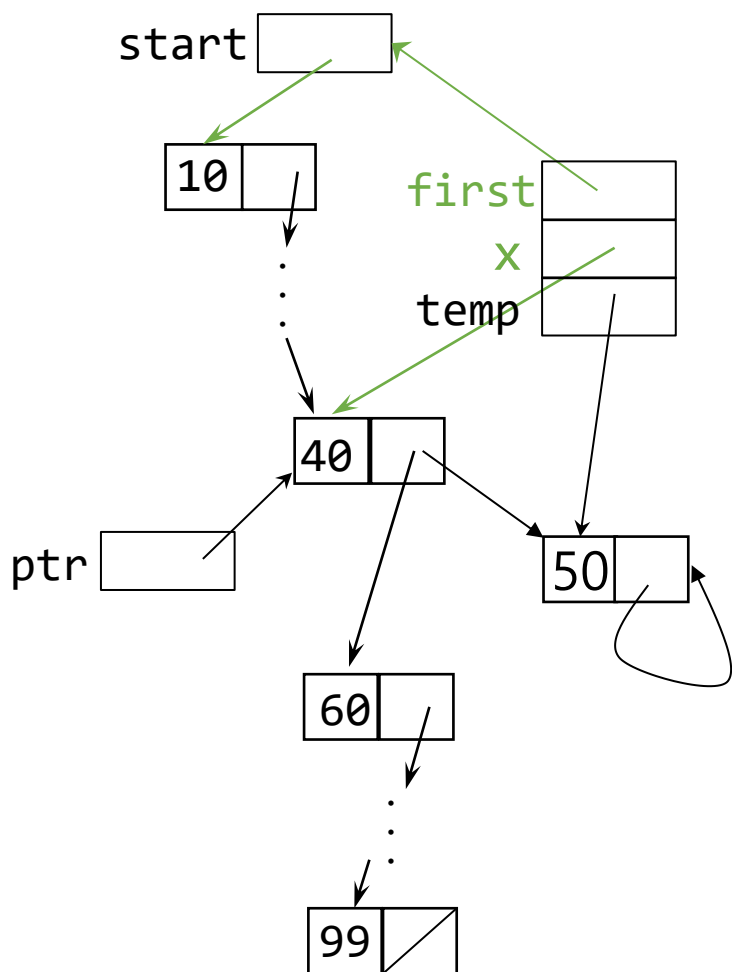
列表插入示例(1)



```
listPointer start, ptr;  
/* 建立列表[list]的代码*/  
insert(&start, ptr);
```

```
void insert(listPointer *first, listPointer x)  
{  
    listPointer temp;  
    MALLOC(temp, sizeof(*temp));  
    temp->data = 50;  
    if(*first) {  
        temp->link = x->link;  
        x->link = temp;  
    }  
    else {  
        /* ... */  
    }  
}
```

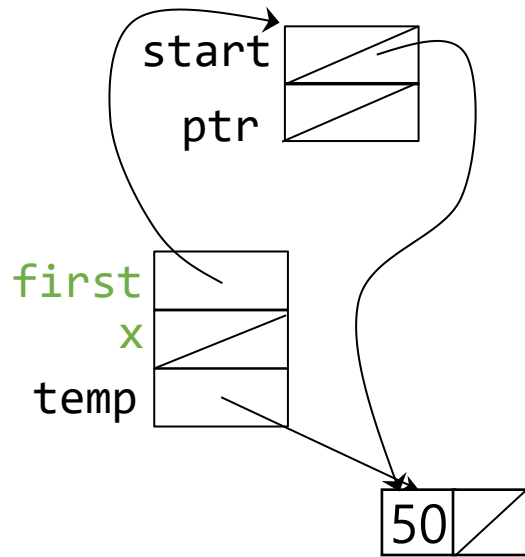
列表插入示例(1)



```
listPointer start, ptr;  
/* 建立列表[list]的代码*/  
insert(&start, ptr);
```

```
void insert(listPointer *first, listPointer x)  
{  
    listPointer temp;  
    MALLOC(temp, sizeof(*temp));  
    temp->data = 50;  
    if(*first) {  
        x->link=temp;    What if??  
        temp->link=x->link;  
    }  
    else {  
        /* ... */  
    }  
}
```

列表插入示例 (2)



```
listPointer start, ptr;  
start=ptr=NULL;  
insert(&start,ptr);
```

```
void insert(listPointer *first, listPointer x)  
{  
    listPointer temp;  
    MALLOC(temp, sizeof(*temp));  
    temp->data = 50;  
    if(*first) { /* ... */  
    } else {  
        temp->link = NULL;  
        *first = temp;  
    }  
}
```

列表删除 (1)

```
void delete(listPointer *first, listPointer trail, listPointer x)
{ /*从链表中删除节点 x, trail 是 x 的前驱节点, *first 是链表的头节点*/
```

```
    if (trail)
```

```
        trail->link = x->link;
```

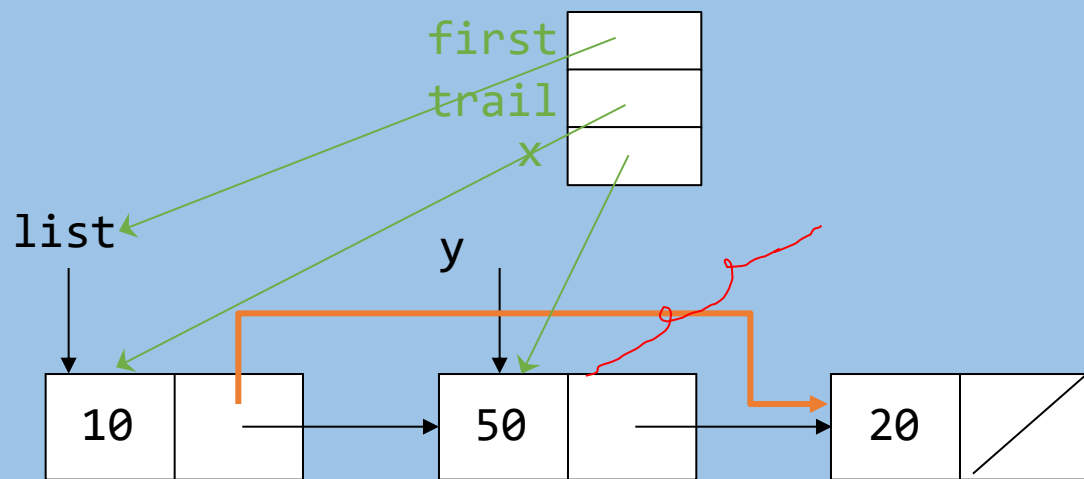
```
    else
```

```
        *first = (*first)->link;
```

```
    free(x);
```

```
}
```

```
delete(&list, list, y);
```



列表删除 (2)

```
void delete(listPointer *first, listPointer trail, listPointer x)
{/*从链表中删除节点 x, trail 是 x 的前驱节点, *first 是链表的头节点*/
```

```
    if (trail)
```

```
        trail->link = x->link;
```

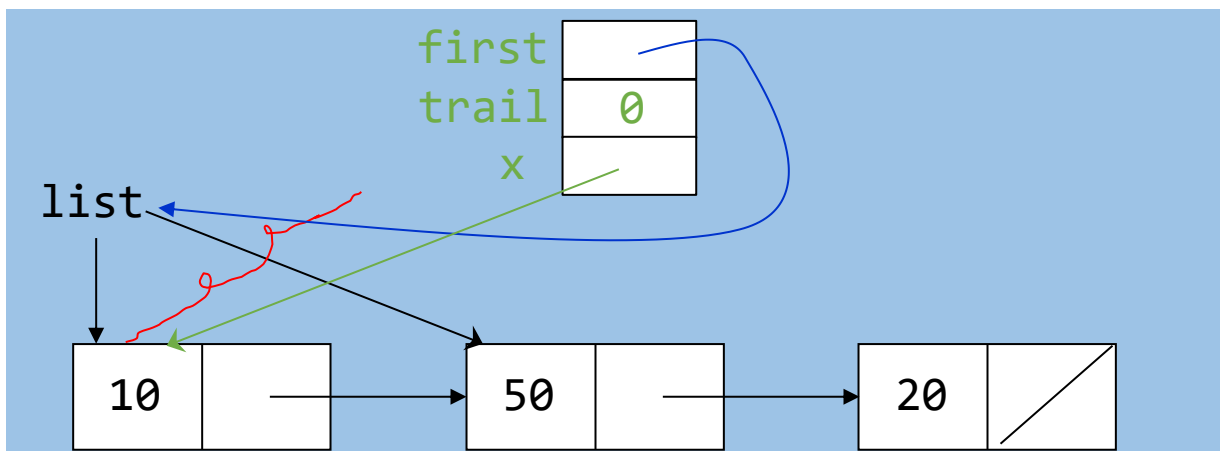
```
    else
```

```
        *first = (*first)->link;
```

```
    free(x);
```

```
}
```

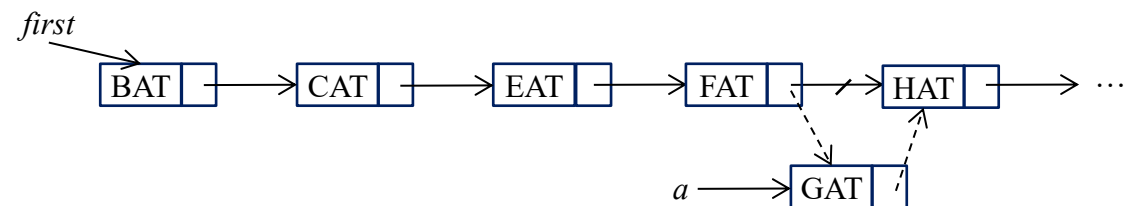
```
delete(&list, NULL, list);
```



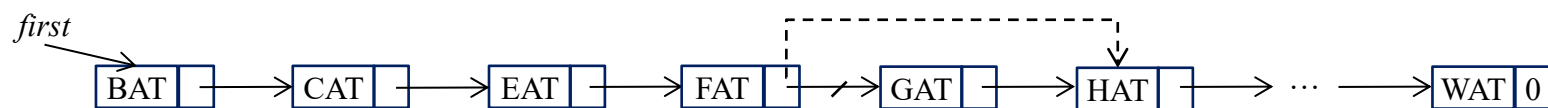
Review

- 链式列表 [Linked list]

Insertion



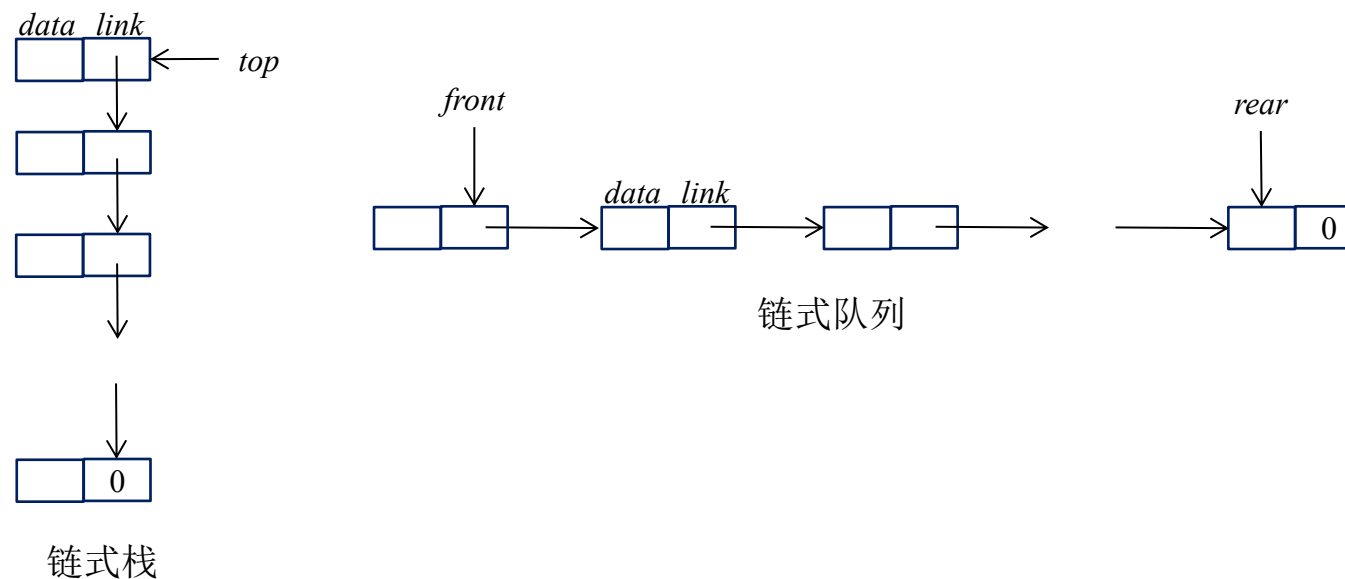
Deletion



```
typedef struct listNode *listPointer;
typedef struct listNode {
    char          data[4];
    listPointer   link;
};
```


链式栈和链式队列

- 当几个堆[stacks]和队列[queues]同时存在时，没有有效的方法来顺序地表达他们
- 链式栈[linked stack]和链式队列[linked queue]
 - 堆栈和队列的链接方向[direction of links]都便于插入和删除节点



n 栈的表示方法

```
#define MAX_STACKS 10 /*最大栈数*/
typedef struct {
    int      key;
    /* other fields */
} element;

typedef struct stack *stackPointer;
typedef struct stack {
    element data;
    stackPointer link;
};

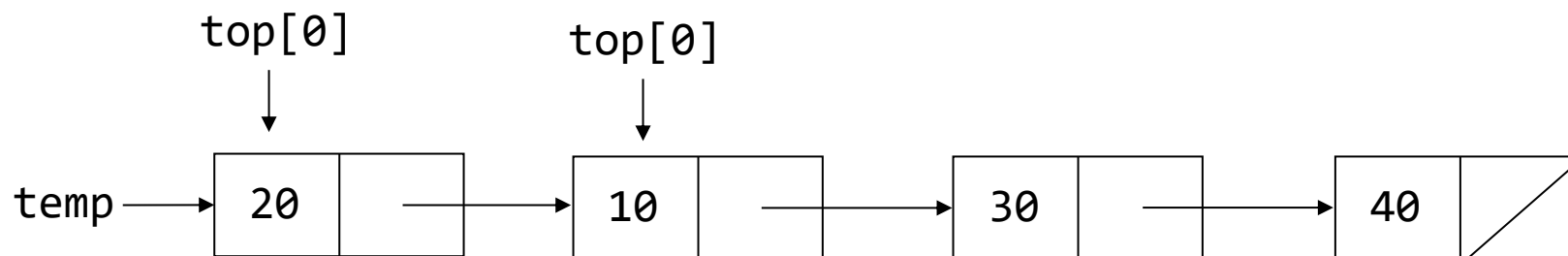
stackPointer top[MAX_STACKS];
```

- 初始化
 - $\text{top}[i] = \text{NULL}$, $0 \leq i < \text{MAX_STACKS}$
- 边界条件
 - $\text{IS_EMPTY}(\text{top}[i])$: 第 i 层栈是空栈（栈中没有元素）
 - $\text{IS_FULL}(\text{temp})$: 没有内存空间

栈操作(1)

```
void push(int i, element item)
{ /* 加入元素[item]到第 i 个栈中*/
  stackPointer temp;
  MALLOC(temp, sizeof (*temp));
  temp->data = item;
  temp->link = top[i];
  top[i] = temp;
}
```

```
element new_item;
new_item.key=20;
push(0, new_item);
```



堆操作 (2)

```
element pop(int i)
{ /* 移除第 i 个栈中的顶层元素*/
```

```
    stackPointer temp = top[i];
```

```
    element item;
```

```
    if (!temp) return stackEmpty();
```

```
    item = temp->data;
```

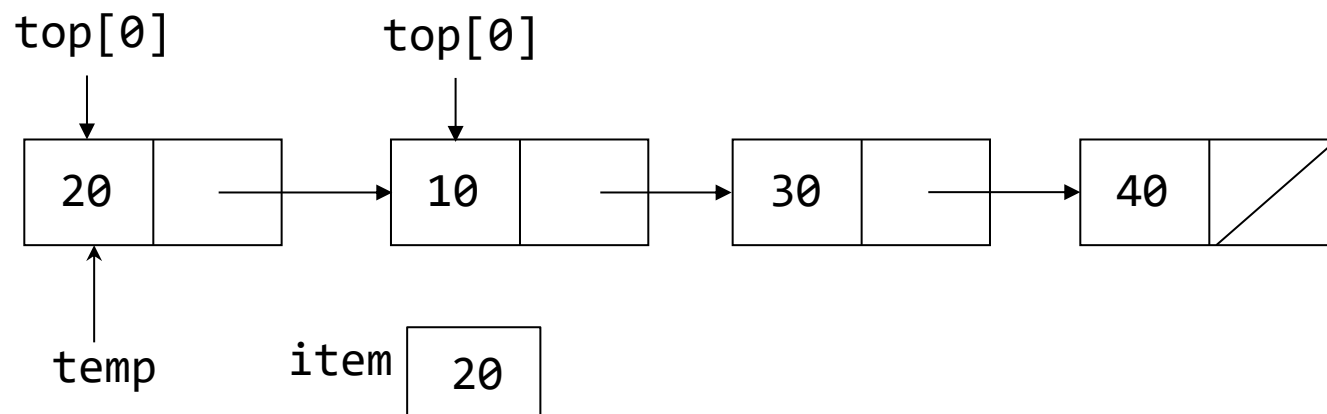
```
    top[i] = temp->link;
```

```
    free(temp);
```

```
    return item;
```

```
}
```

```
item = pop(0);
printf("%d\n", item.key);
```



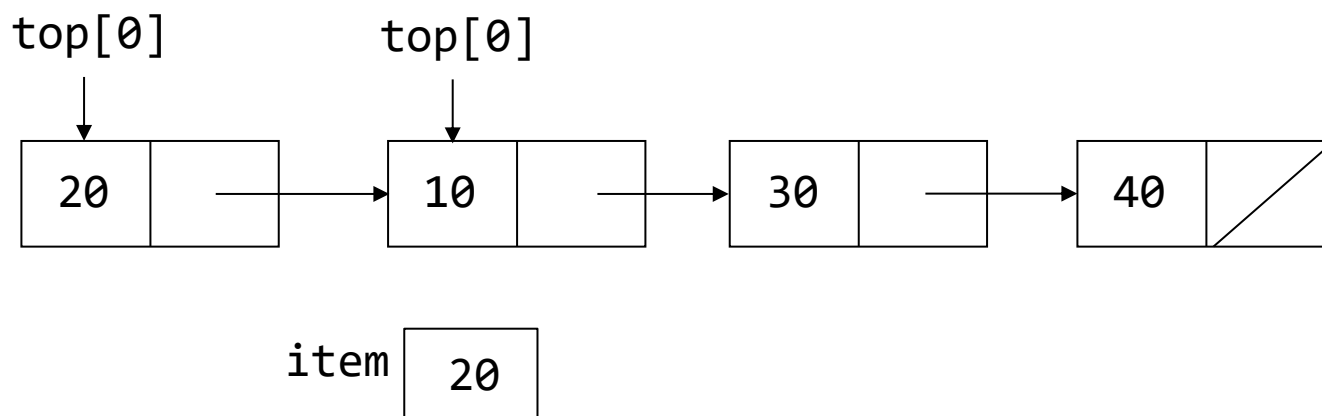
堆操作 (2)

```
element pop(int i)
{ /*移除第 i 个栈中的顶层元素*/

    element item;
    if (!top[i]) return stackEmpty();
    item = top[i]->data;
    top[i] = top[i]->link;
    return item;
}
```

如果??

```
element data;
data = pop(0);
printf("%d\n", data.key);
```



多队列[queue]

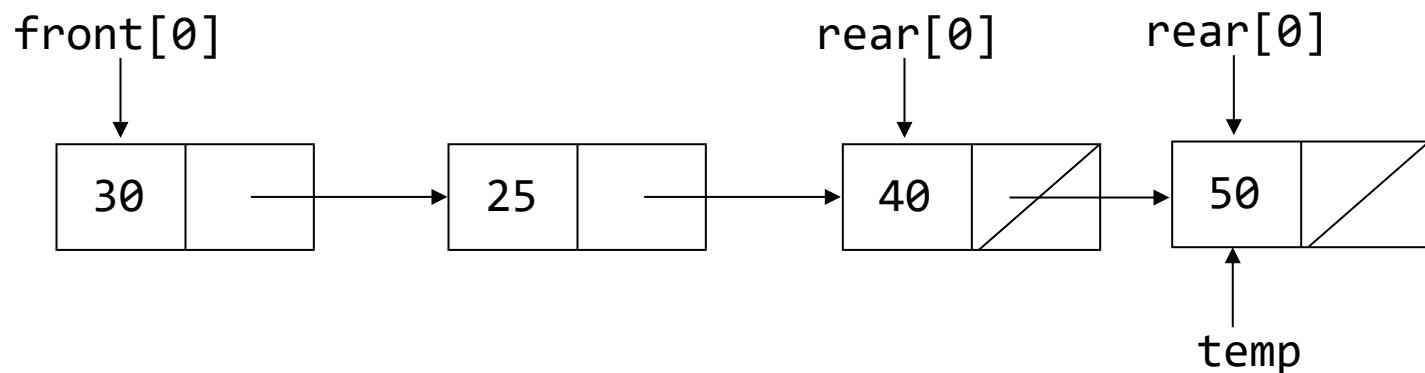
```
#define MAX_QUEUE 10    /* 最大队列数*/  
typedef struct queue* queuePointer;  
typedef struct queue {  
    element data;  
    queuePointer link;  
};  
queuePointer front[MAX_QUEUE], rear[MAX_QUEUE];
```

- 初始条件
 - $\text{front}[\mathbf{i}] = \text{NULL}$, $0 \leq \mathbf{i} < \text{MAX_QUEUES}$
- 边界条件
 - $\text{IS_EMPTY}(\text{front}[\mathbf{i}])$: no item is available in $\mathbf{i}$ -th queue
 - $\text{IS_FULL}(\text{temp})$: no memory space is available

队列操作(1)

```
void addq (int i, element item)
{ /*在队列 i 后插入一个元素*/
  queuePointer temp;
  MALLOC(temp, sizeof(*temp));
  temp->data = item;
  temp->link = NULL;
  if (front[i]) rear[i]->link = temp;
  else front[i] = temp;
  rear[i] = temp;
}
```

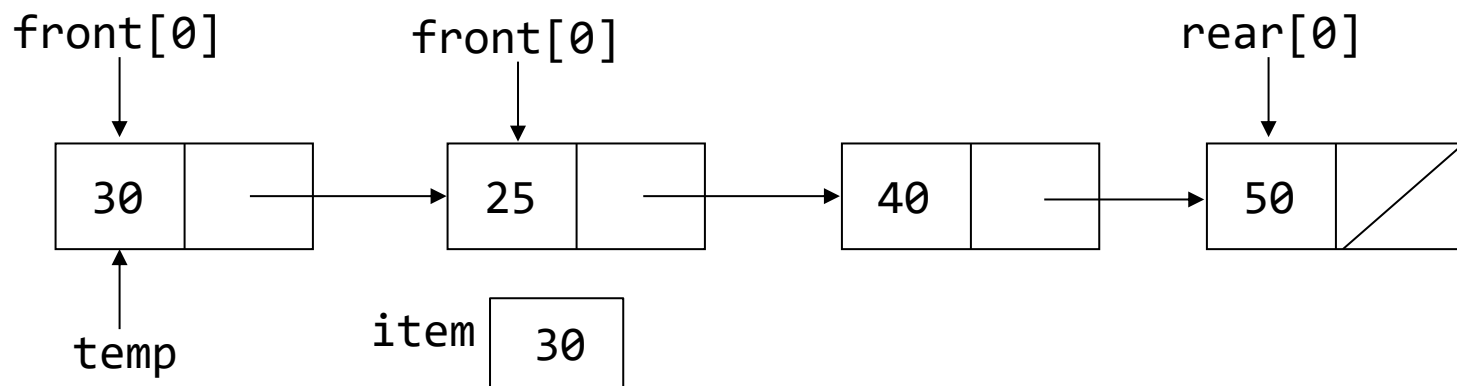
```
element item;
item.key=50;
addq(0, item);
```



队列操作 (2)

```
element deleteq (int i)
{ /* 在队列 i 中删除一个元素*/
    queuePointer temp = front[i];
    element item;
    if (!temp) return queueEmpty();
    item = temp->data;
    front[i] = temp->link;
    free(temp);
    return item;
}
```

```
item=deleteq(0);
printf("%d\n", item.key);
```



多项式 [Polynomials]

- 用链表表达多项式

$$A(x) = a_{m-1}x^{e_{m-1}} + \cdots + a_0x^{e_0}$$

其中 a_i 是非零系数, e_i 是非负整数指数, 以致 $e_{m-1} > e_{m-2} > \cdots > e_1 > e_0 \geq 0$

- 我们将每个项表示为一个节点 [term], 节点包含系数[coefficient]和指数[exponent]字段, 以及一个指向下一项的指针

多项式的表达

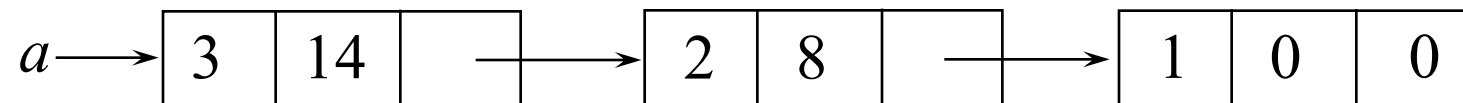
- 多项式项的声明

```
typedef struct polyNode *polyPointer;  
typedef struct polyNode {  
    int coef;  
    int expon;  
    polyPointer link;  
};  
polyPointer a,b,c;
```

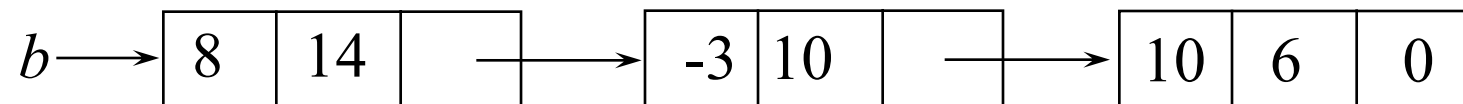
coef	expon	link
------	-------	------

多项式 [Polynomials]

$$a = 3x^{14} + 2x^8 + 1$$



$$b = 8x^{14} - 3x^{10} + 10x^6$$



两个多项式相加(1)

```
polyPointer padd (polyPointer a, polyPointer b){
    /*返回 a 和 b 之和的多项式*/
    polyPointer c, rear, temp; int sum;
    MALLOC (rear, sizeof(*rear));
    c = rear;
    while(a && b) {
        switch (COMPARE(a->expon, b->expon)) {
            case -1: /* a->expon < b->expon */
                attach(b->coef,b->expon,&rear);
                b = b->link; break;
            case 0: /* a->expon = b->expon */
                sum = a->coef + b->coef;
                if (sum) attach(sum,a->expon,&rear);
                a = a->link; b=b->link; break;
            case 1: /* a->expon > a->expon */
                attach(a->coef,a->expon,&rear);
                a = a->link;}
    }
```

两个多项式相加 (2)

```
/*复制列表 a 和列表 b 的其余部分*/  
for (; a; a = a->link) attach(a->coef,a->expon,&rear);  
for (; b; b = b->link) attach(b->coef,b->expon,&rear);  
rear->link = NULL;
```

```
/*删除无用的初始节点*/  
temp = c;    c = c->link;    free(temp);  
return c;
```

```
}
```

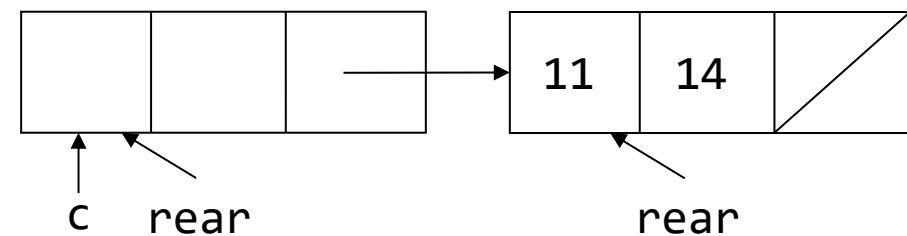
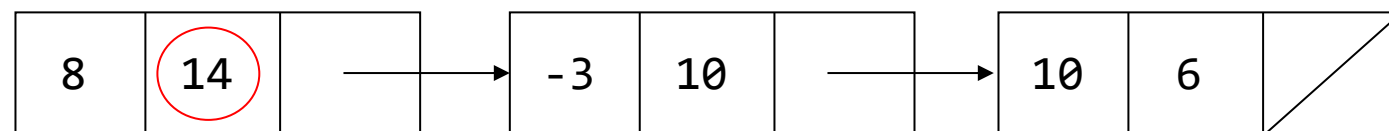
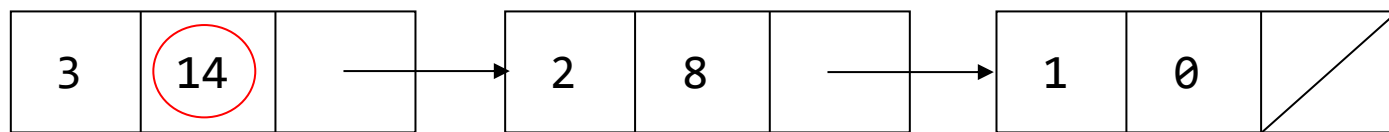
两个多项式相加 (3)

```
void attach(float coefficient, int exponent, polyPointer *ptr)
{
    /* 创建一个新节点 coef = coefficient 并且 expon = exponent, 并将该
       节点连接到 ptr 所指向的节点。然后更新 ptr使其指向    这个新节点*/

    polyPointer temp;
    MALLOC( temp, sizeof(*temp) );
    temp->coef = coefficient;
    temp->expon = exponent;
    (*ptr)->link = temp;
    *ptr = temp;
}
```

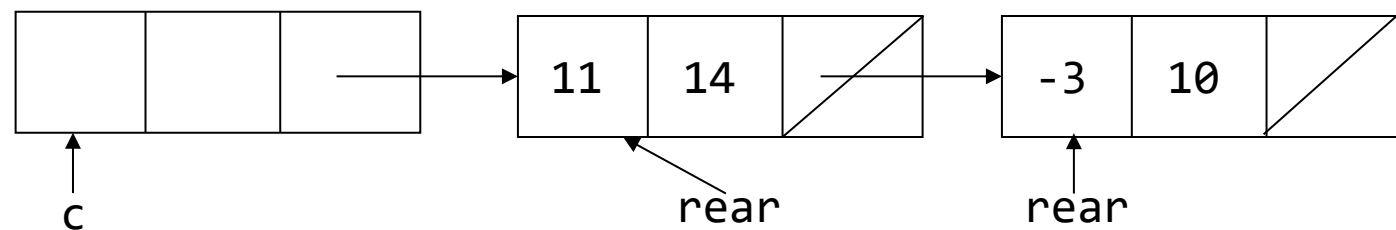
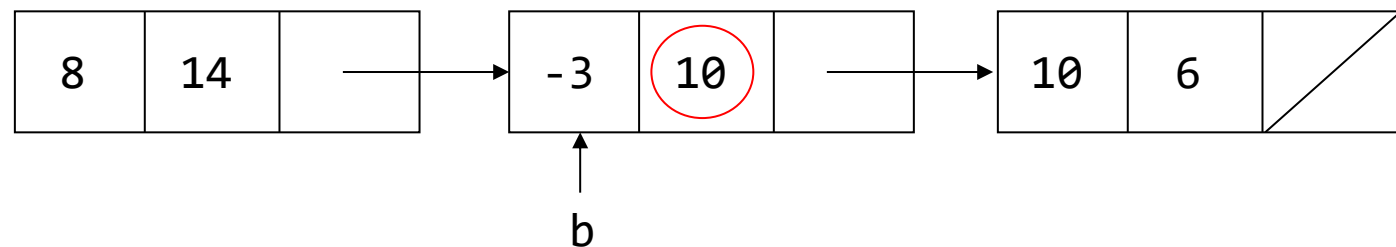
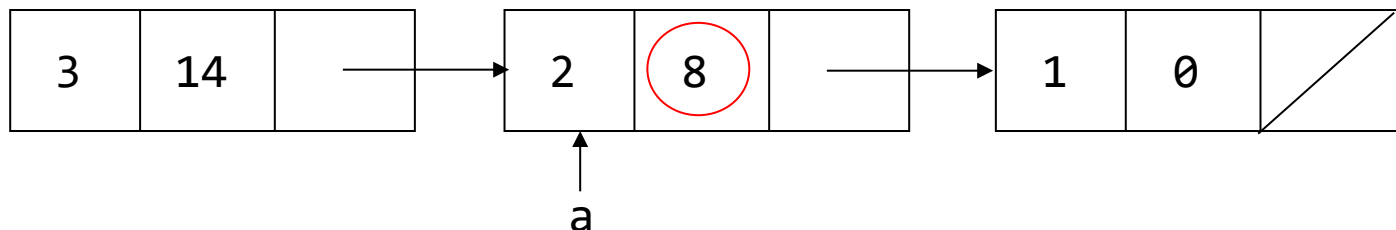
多项式 - 相加 (1)

```
if (a->expon == b->expon)
    attach(a->coef + b->coef, a->expon, &rear);
```



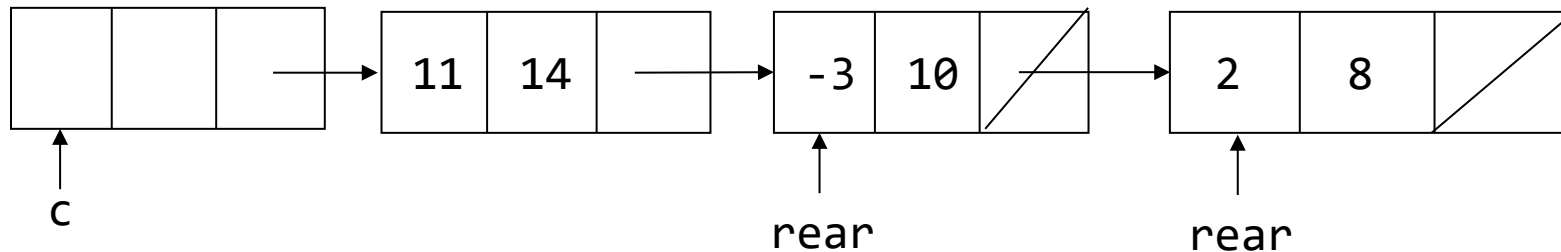
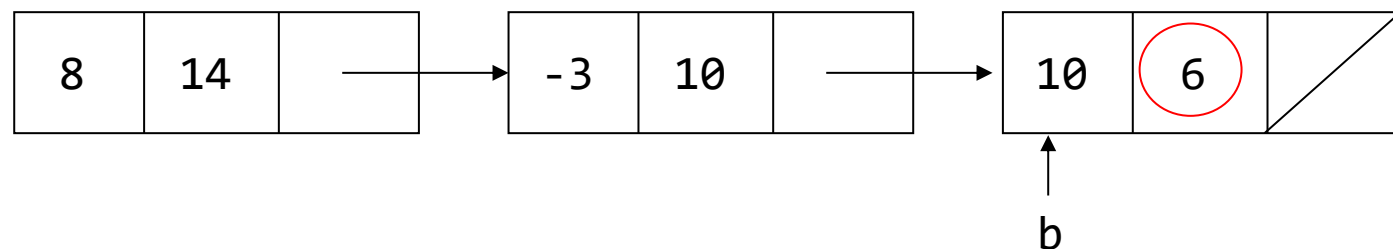
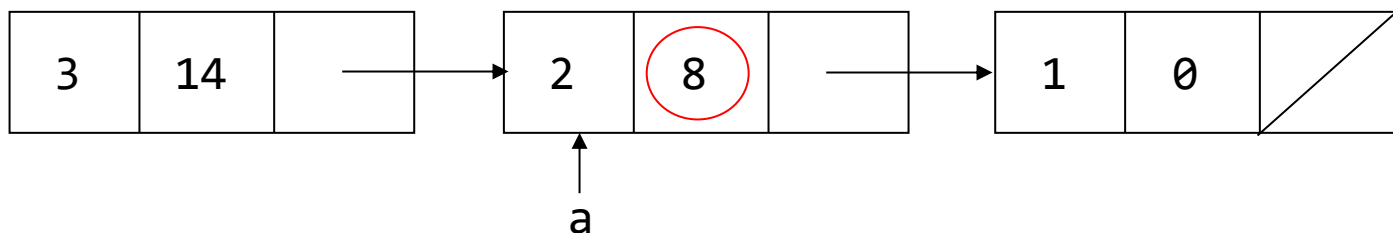
多项式 – 相加(2)

```
if (a->expon < b->expon)  
    attach(b->coef, b->expon, &rear);
```



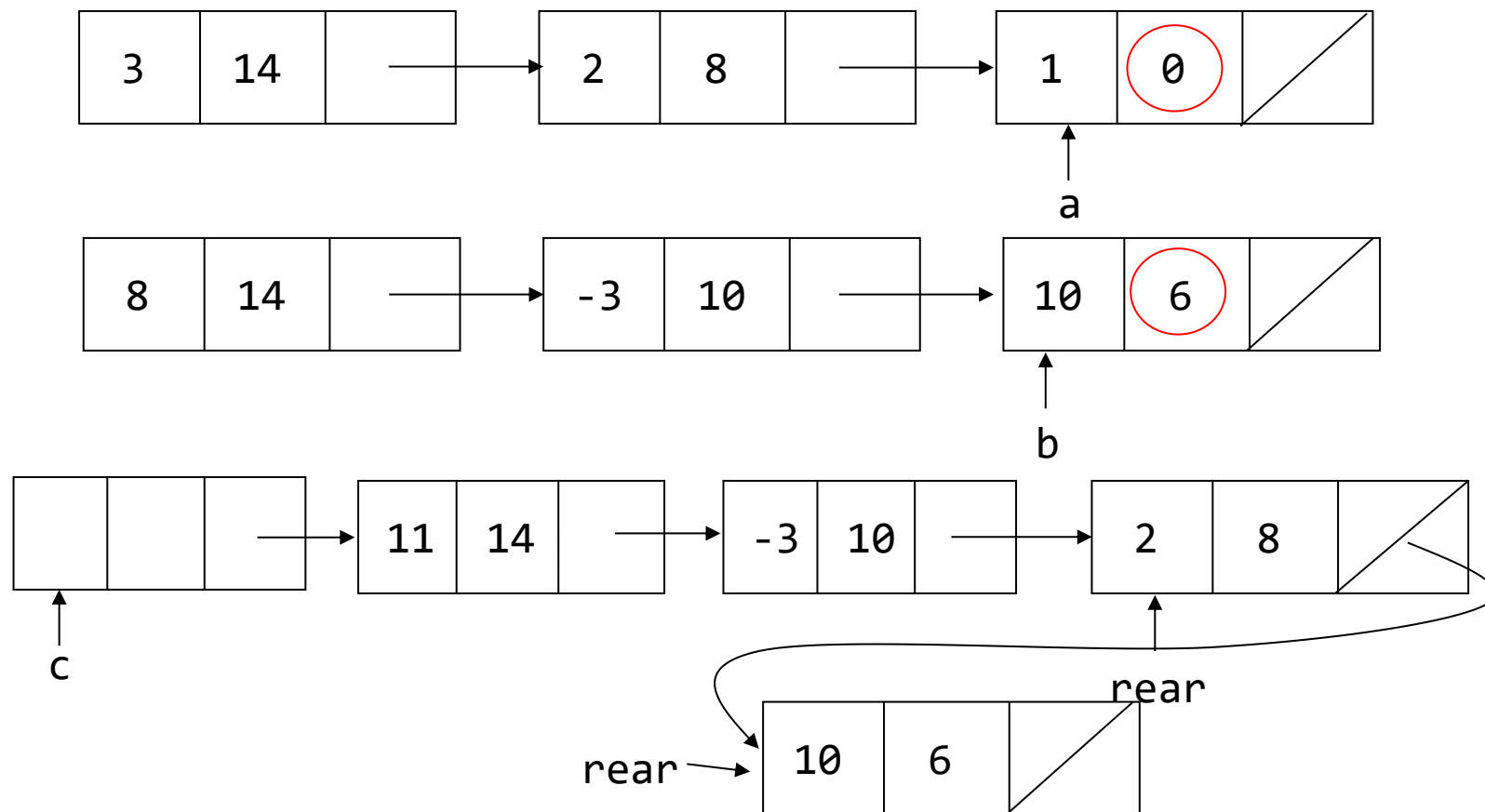
多项式 – 相加(3)

```
if (a->expon > b->expon)
    attach(a->coef, a->expon, &rear);
```



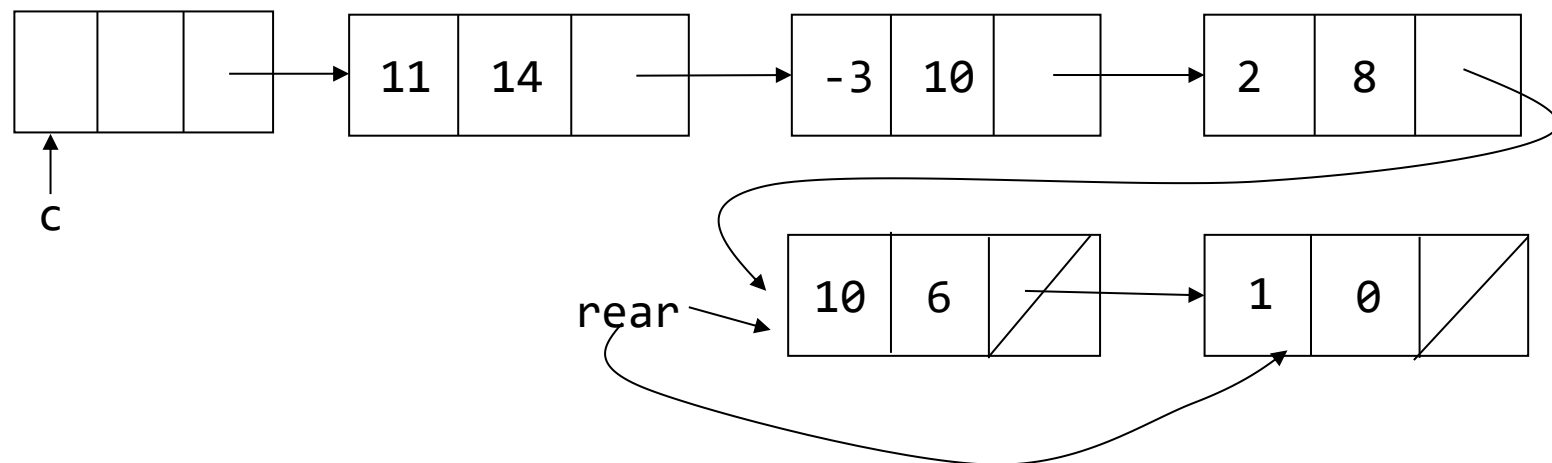
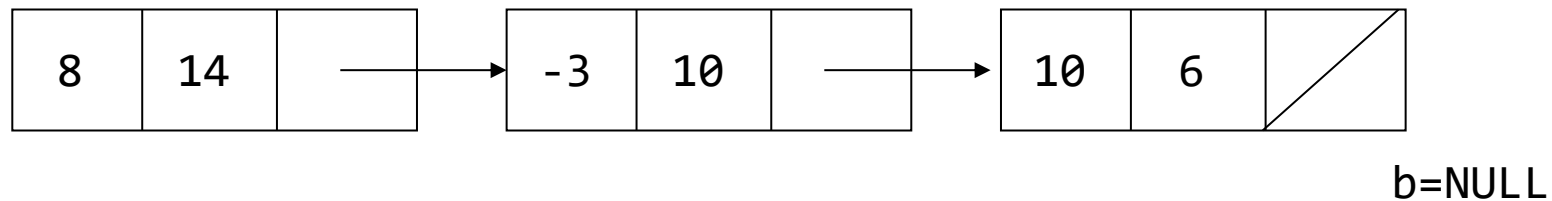
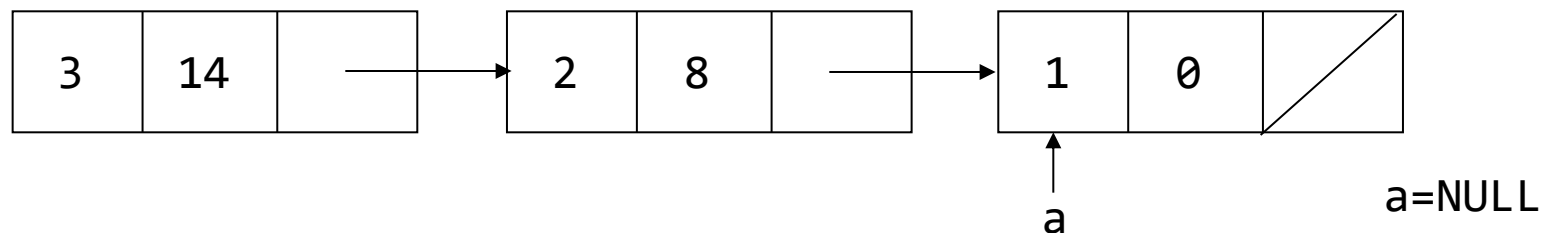
多项式 – 相加(4)

```
if (a->expon < b->expon)
    attach(b->coef, b->expon, &rear);
```



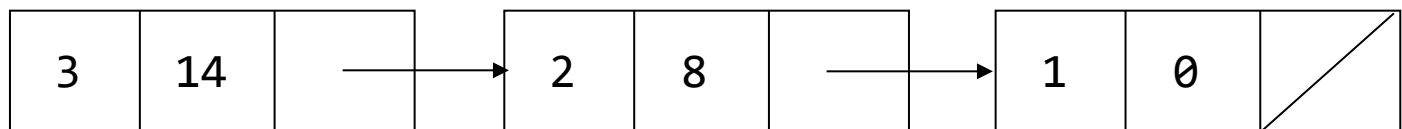
多项式 - 相加(5)

```
for (; a; a = a->link)
    attach(a->coef, a->expon, &rear);
```

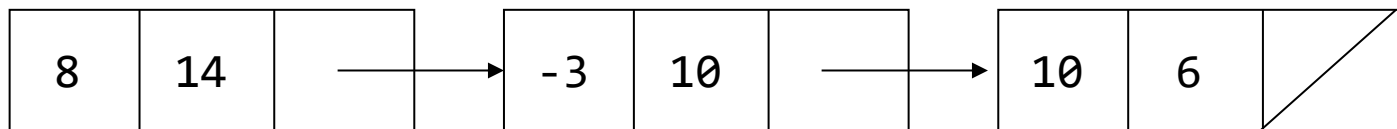


多项式 - 相加(6)

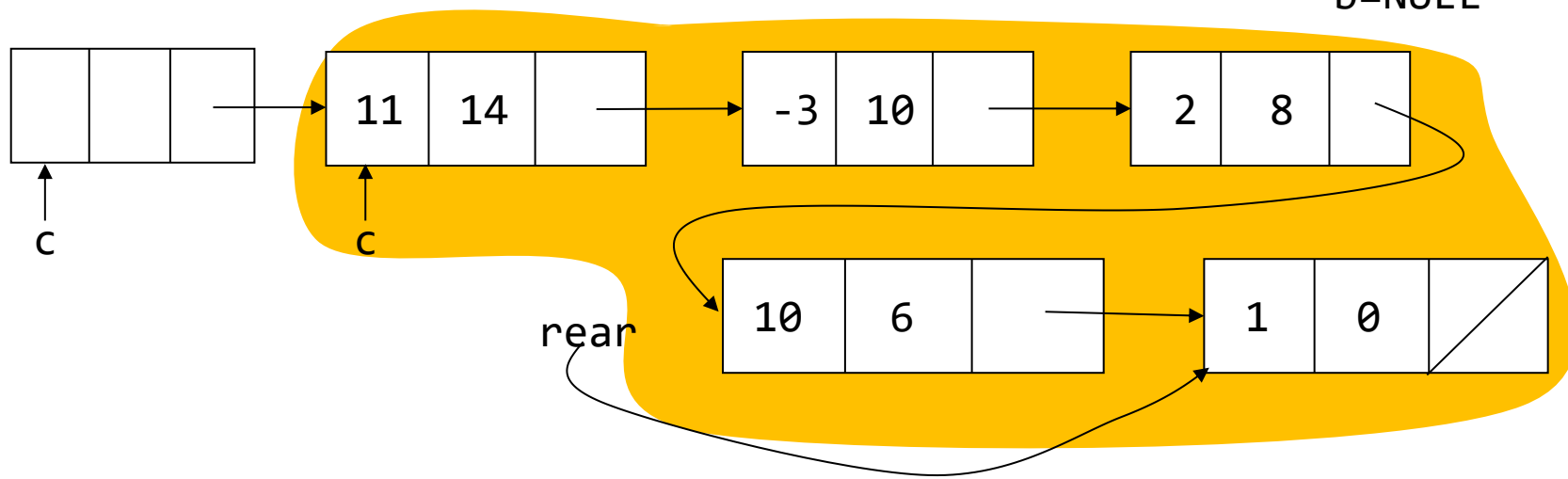
```
c=c->link;  
return c;
```



a=NULL



b=NULL



多项式加法分析

$$A(x) = a_{m-1}x^{e_{m-1}} + \dots + a_0x^{e_0}$$

$$B(x) = b_{n-1}x^{f_{n-1}} + \dots + b_0x^{f_0}$$

其中 $a_i, b_i \neq 0, e_{m-1} > \dots > e_0 \geq 0, f_{n-1} > \dots > f_0 \geq 0,$

$$0 \leq \text{系数加法的次数} \leq \min\{m, n\}$$

$$\text{项数} \leq m + n$$

$$\rightarrow \text{指数比较次数} \leq m + n$$

$$\rightarrow f(m, n) = O(m+n)$$

合并同类项

❖ 计算 $e(x) = a(x) * b(x) + d(x)$

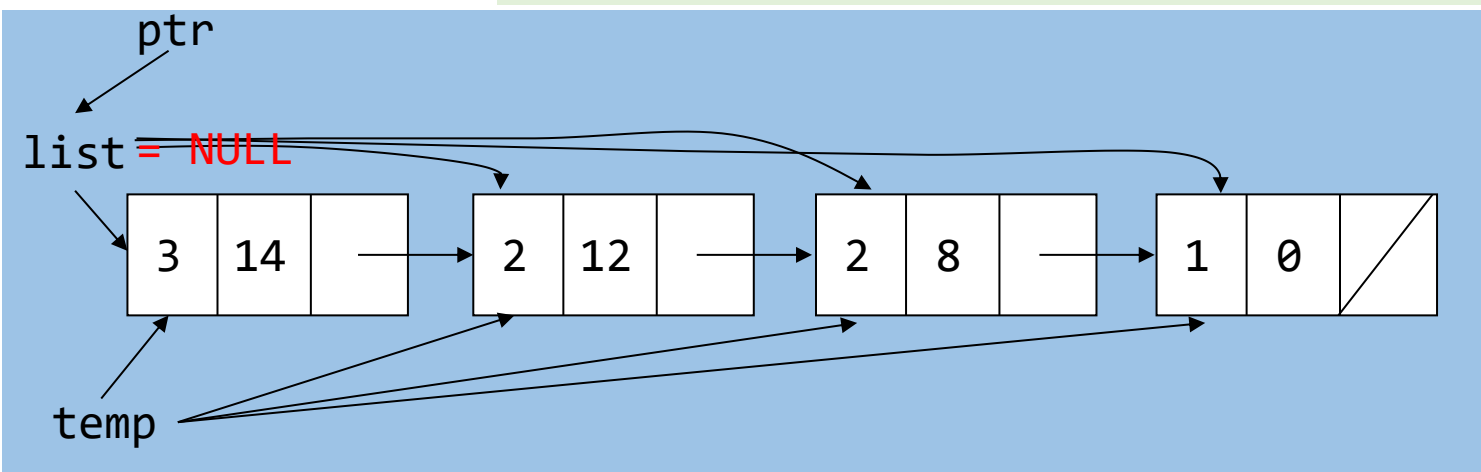
```
polyPointer a, b, d, e;  
...  
a = readPoly();  
b = readPoly();  
d = readPoly();  
temp = pmult(a, b); /* only hold a partial result for d(x) */  
e = padd(temp, d);  
printPoly(e);
```

- We created *temp(x)* only to hold a partial result for *d(x)*
 - It would be useful to reclaim the nodes that are being used to represent *temp(x)*

多项式消解

```
void erase(polyPointer *ptr)
{
    /*消除 ptr 指向的多项式*/
    polyPointer temp;
    while (*ptr)
    {
        temp = *ptr;
        *ptr = (*ptr)→link;
        free(temp);
    }
}
```

erase (&list);

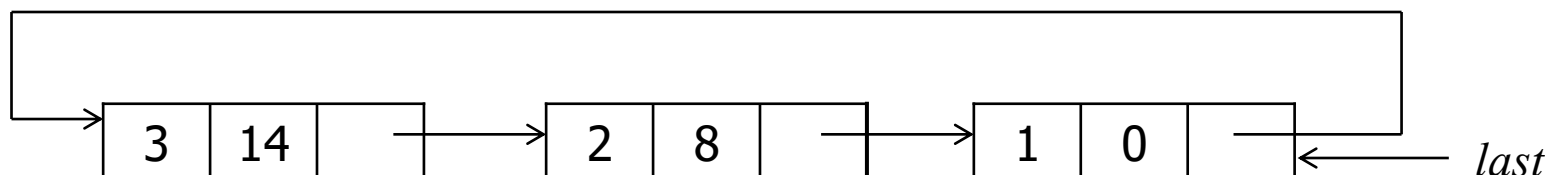


多项式的循环链表表示

[Circular List Representation of Polynomials]

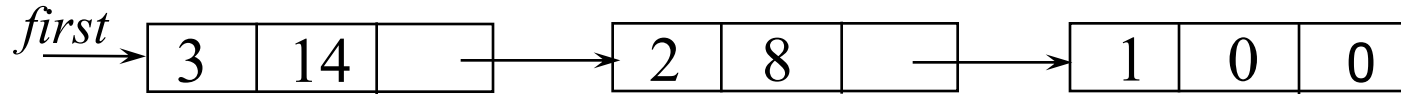
- ❖ 如果最后一个节点的链接字段指向链表中的第一个节点，那么可以更高效地释放多项式的所有节点
- ❖ 循环表示

$$3x^{14} + 2x^8 + 1$$



可用空间链表

- 链表
 - 单链表的最后一个节点的链接为空



- 一个用于循环列表的高效的擦除算法, 通过维护一个由已‘释放’节点组成的链表（作为链式结构）
 - 当需要新节点时，检查这个链表
 - 如果链表不为空，则可以使用其中的一个节点
 - 只有当链表为空时，才需要使用 `malloc` 创建一个新节点
- 这个链表被称为可用空间列表（available space list）或空闲列表（avail list）
 - 初始设置 *avail* 为 `NULL`
- 替代用 *malloc* 和 *free*, 现在用 *getNode* 和 *retNode*

getNode 函数

```
polyPointer getNode(void)
{
    /* 提供一个用于使用的节点*/
    polyPointer node;
    if ( avail ) {
        node = avail;
        avail = avail->link;
    }
    else
        MALLOC( node, sizeof(*node) );

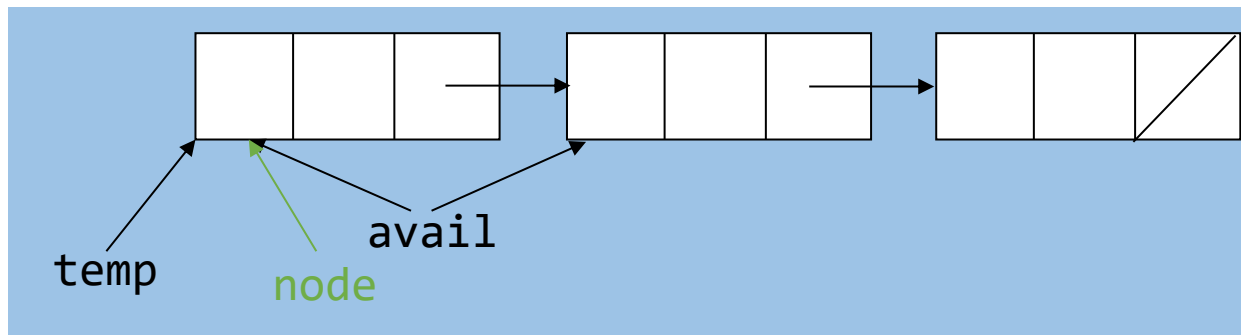
    return node;
}
```

getNode 函数

```
polyPointer getNode(void)
{ /* 提供一个用于使用的节点 */
  polyPointer node;
  if (avail) {
    node = avail;
    avail = avail->link;
  }
  else MALLOC(node, sizeof(*node));
  return node;
}
```

```
polyPointer avail; /*全局变量*/
```

```
polyPointer temp=getNode();
```



getNode 函数

```
polyPointer getNode(void)
{ /* 提供一个用于使用的节点 */
  polyPointer node;
  if (avail) {
    node = avail;
    avail = avail->link;
  }
  else MALLOC(node, sizeof(*node));
  return node;
}
```

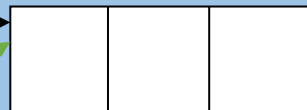
```
polyPointer avail; /*全局变量*/
```

```
polyPointer temp=getNode();
```

avail=NULL

temp

node



retNode 函数

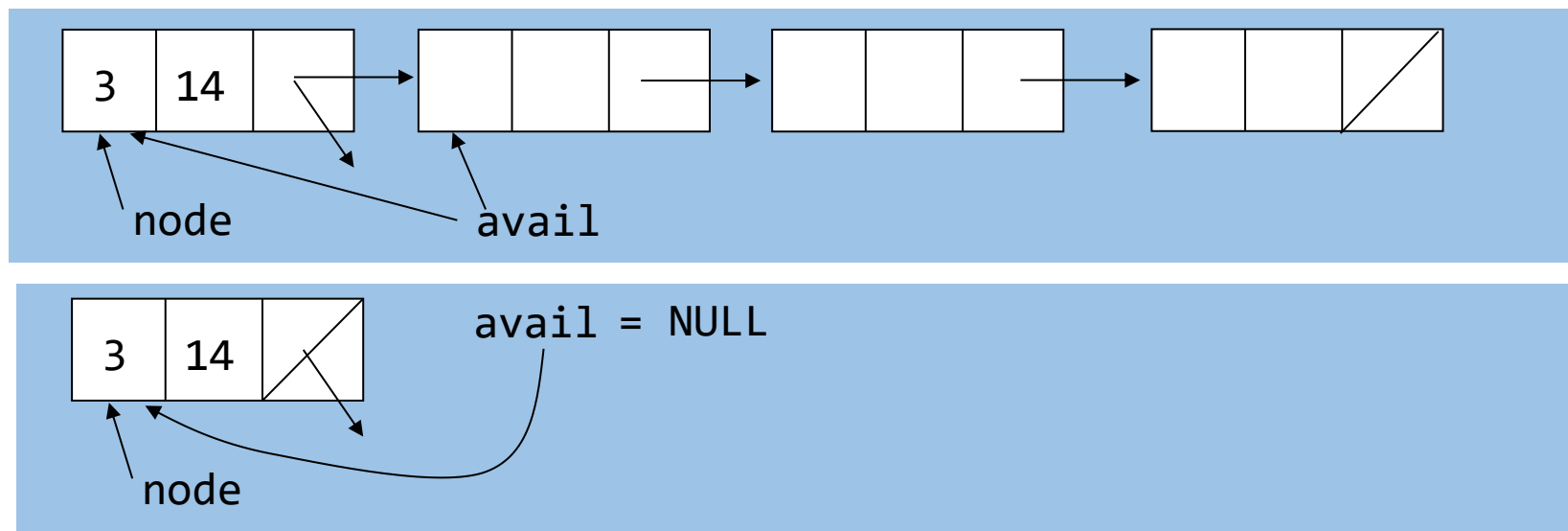
❖ retNode: 将一个节点返回可用列表

```
void retNode(polyPointer node)
{
    node->link = avail;
    avail = node;
}
```

retNode 函数

```
void retNode(polyPointer node)
{ /* 返回可用列表中的一个节点 */
    node->link = avail;
    avail = node;
}
```

```
polyPointer avail;
/*一个指向空闲节点链表第一个节点的全局变量*/
```



Cerase 函数

❖ **cerase**:在固定时间内删除循环链表，且时间与链表中的节点数量无关

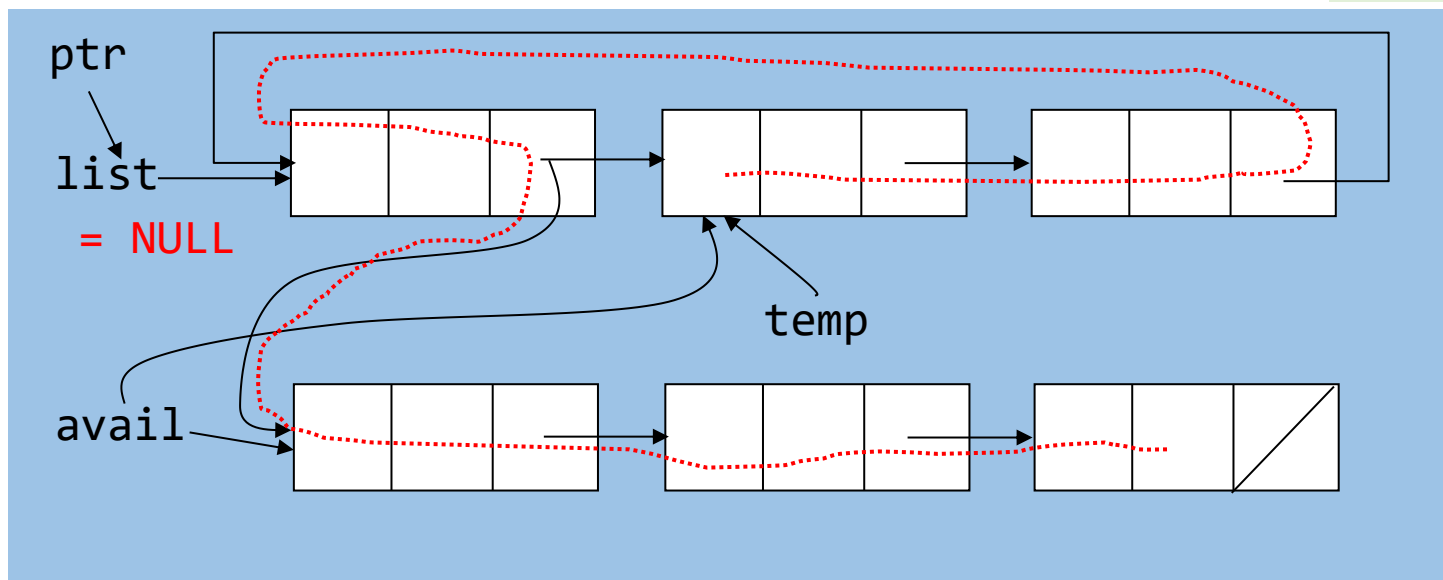
```
void cerase( polyPointer *ptr )
{
    /*擦除 ptr 指向的循环列表*/
    polyPointer temp;
    if (*ptr) {
        temp = (*ptr)→link;
        (*ptr)→link = avail;
        avail = temp;
        *ptr = NULL;
    }
}
```

擦除循环列表

```
void cerase(polyPointer* ptr)
{ /* /*擦除 ptr 指向的循环列表*/
  */
  polyPointer temp;
  if (*ptr) {
    temp = (*ptr)->link;
    (*ptr)->link = avail;
    avail = temp;
    *ptr = NULL;
  }
}
```

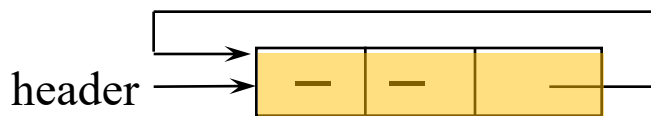
```
polyPointer avail;
"/ 一个指向空闲节点链表第一个节点的
全局变量  /"
```

```
cerase(&list);
```

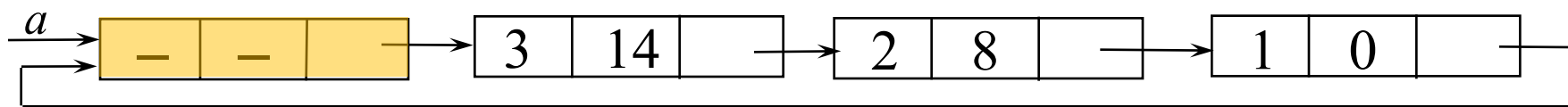


零多项式

- ❖ 为了防止多项式系数为零的特殊情况出现，每个多项式后面包含一个额外节点，一个头节点 [header node]
 - 该节点的指数 (expon) 和系数 (coef) 字段无关紧要
- ❖ 表示方法
 - 零多项式



$$a = 3x^{14} + 2x^8 + 1$$



❖两个多项式以循环列表的形式表示，带标题节点

```
polyPointer cpadd(polyPointer a, polyPointer b)
{
    /* 多项式 a 和 b 是带有头节点单循环链表，返回的多项式是a and b 的和*/
    polyPointer startA, c, lastC;
    int sum, done = FALSE;
    startA = a;          /* 记录 a 的起始位置*/
    a = a->link;          /* 跳过 a 和 b 的头节点*/
    b = b->link;
    c = getNode();        /*为 sum 获取一个头节点*/
    c->expon = -1;
    lastC = c;
    do {
        switch (COMPARE(a->expon, b->expon))
        {
            case -1:      /* a->expon < b->expon */
                attach(b->coef, b->expon, &lastC);
                b = b->link;
                break;
        }
    }
}
```

```
        case 0:      /* a→expon = b→expon */
            if (startA == a)
                done = TRUE;
            else {
                sum = a→coef + b→coef;
                if (sum)    attach(sum, a→expon, &lastC);
                a = a→link;    b = b→link;
            }
            break;

        case 1: /* a→expon > b→expon */
            attach(a→coef, a→expon, &lastC);
            a = a→link;
    }
} while ( !done );

lastC→link = c;
return c;
}
```

额外链表操作

❖ 链表操作

- invert()

- concatenate()

❖ 循环链表的操作

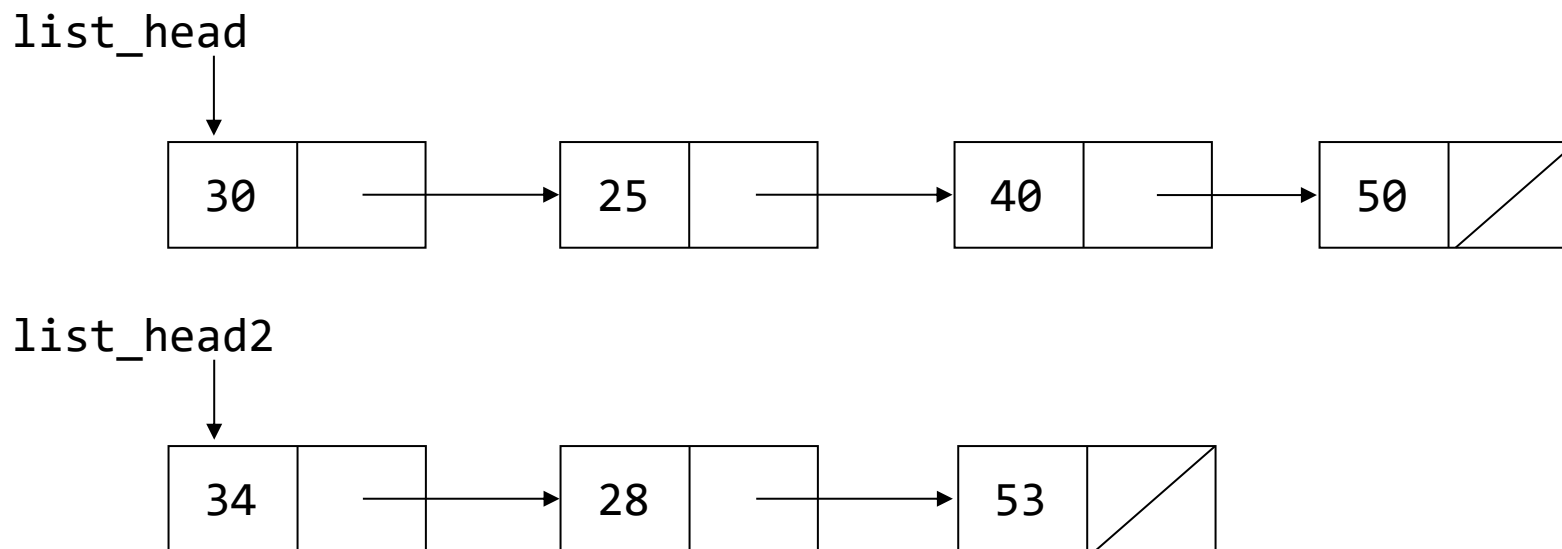
- insertFront()

- length()

额外链表操作

❖ 基于数据结构

```
typedef struct listNode* listPointer;  
typedef struct listNode {  
    int data;  
    listPointer link;  
};  
listPointer list_head, list_head2;
```

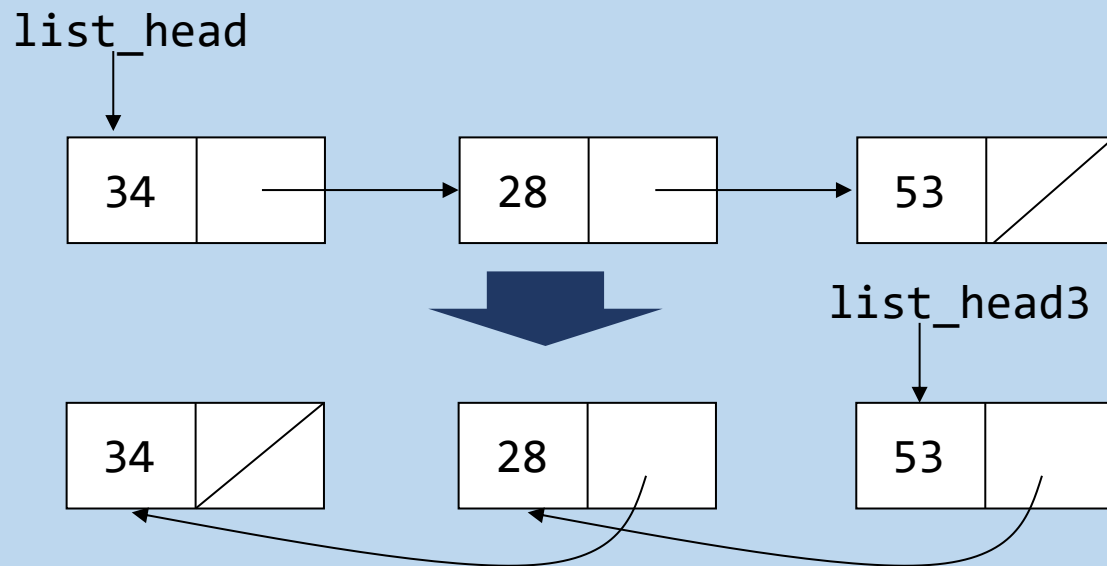


额外链表操作

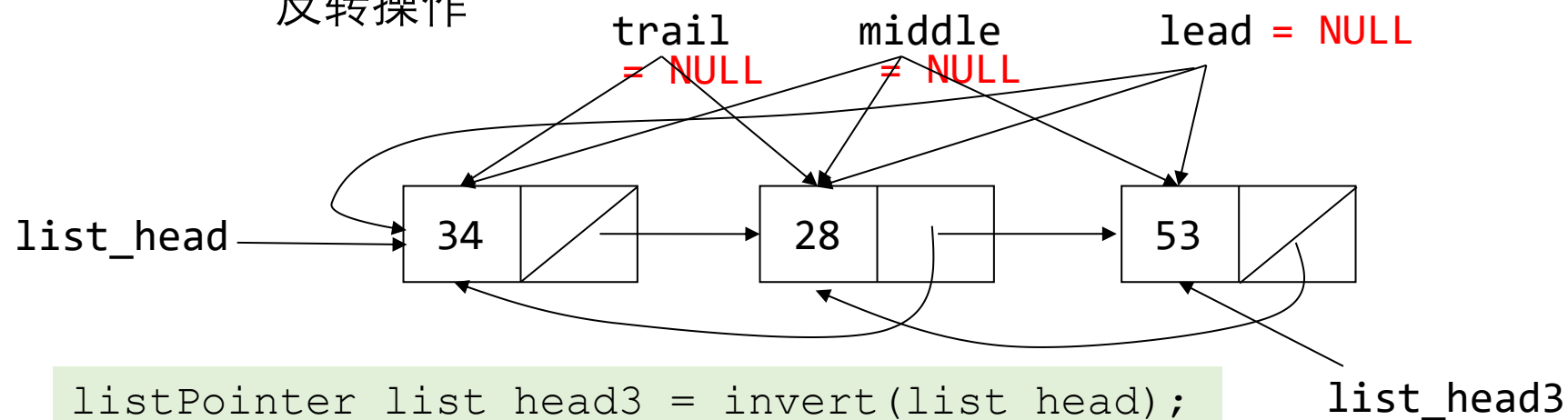
❖ 反转

```
listPointer invert(listPointer lead)
{
    listPointer middle, trail;
    middle = NULL;
    while (lead) {
        trail = middle;
        middle = lead;
        lead = lead->link;
        middle->link = trail;
    }
    return middle;
}
```

```
listPointer list_head3 = invert(list_head);
```



反转操作



```
listPointer list_head3 = invert(list_head);
```

```
listPointer invert(listPointer lead)
```

```
{
```

```
listPointer middle, trail;
```

```
middle = NULL;
```

```
while (lead) {
```

```
    trail = middle;
```

```
    middle = lead;
```

```
    lead = lead->link;
```

```
    middle->link = trail;
```

```
}
```

```
return middle;
```

```
}
```

额外链表操作

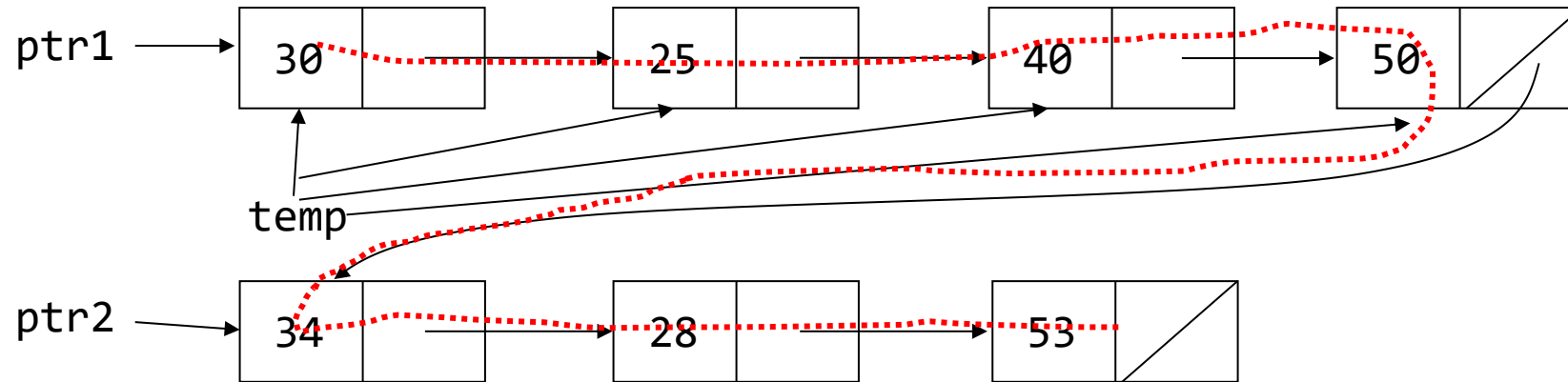
❖ 连接

```
listPointer concatenate (listPointer ptr1, listPointer ptr2)
{ /*生成一个新列表, 其中包含ptr1
   followed by the list ptr2 */
   listPointer temp;

   if (!ptr1)      return ptr2;
   if (!ptr2)      return ptr1;

   /* find end of first list */
   for (temp = ptr1; temp->link; temp = temp->link);

   temp->link = ptr2;    /*首尾相接*/
   return ptr1;
}
```

```
ptr1 = concatenate (ptr1, ptr2);
```

```
listPointer concatenate (listPointer ptr1, listPointer ptr2)
{
    listPointer temp;
    if (!ptr1) return ptr2;
    if (!ptr2) return ptr1;

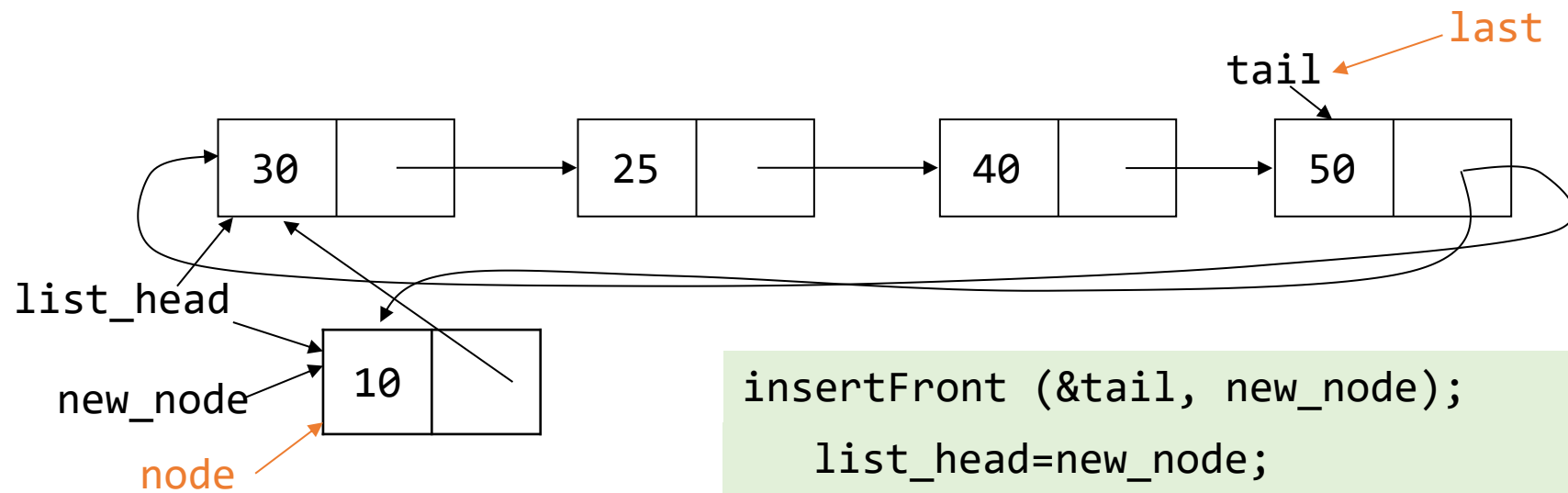
    for (temp = ptr1; temp->link; temp = temp->link);

    temp->link = ptr2;
    return ptr1;
}
```

额外链表操作

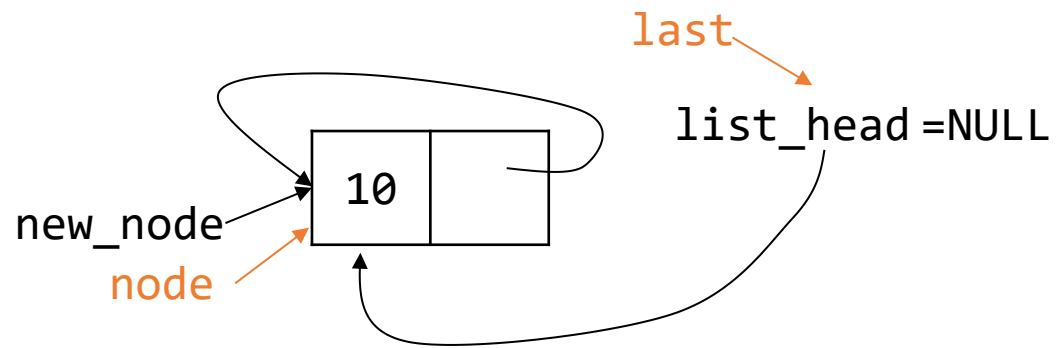
❖ 前置插入 循环链表[circular list]

```
void insertFront (listPointer* last, listPointer node)
{
    /*在循环列表前插入节点, 其最后一个节点由 *last 指向*/
    if (!(*last)) {
        *last = node;
        node->link = node;
    } else {
        node->link = (*last)->link;
        (*last)->link = node;
    }
}
```



```
insertFront (&tail, new_node);  
list_head=new_node;
```

```
void insertFront(listPointer *last, listPointer node)  
{  
    if (!(*last)) {  
        *last = node;  
        node->link = node;  
    } else {  
        node->link = (*last)->link;  
        (*last)->link = node;  
    }  
}
```



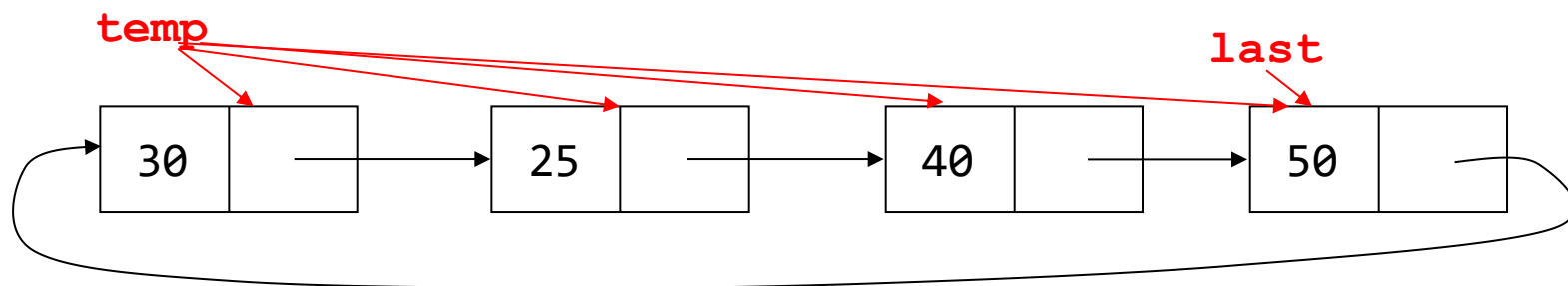
```
listPointer list_head=NULL;  
insertFront (&list_head, new_node);
```

```
void insertFront (listPointer *last, listPointer node)  
{  
    if (!(*last)) {  
        *last = node;  
        node->link = node;  
    } else {  
        node->link = (*last)->link;  
        (*last)->link = node;  
    }  
}
```

额外链表操作

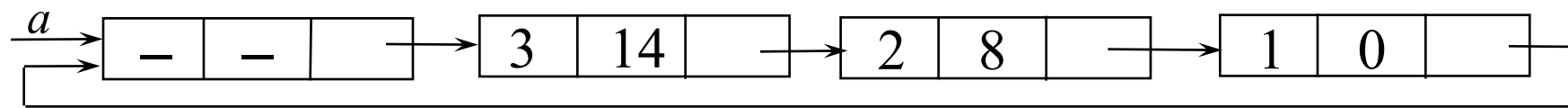
❖ 寻找长度 (循环链表)

```
int length (listPointer last)
{
    listPointer temp;
    int count = 0;
    if (last) {
        temp = last;
        do {
            count++;
            temp = temp->link;
        } while (temp != last);
    }
    return count;
}
```



为什么使用双向链表

目前为止，我们用链表，单链表和循环链表，**限制性高！**



例如, 删除任意节点需要知道其前驱节点.

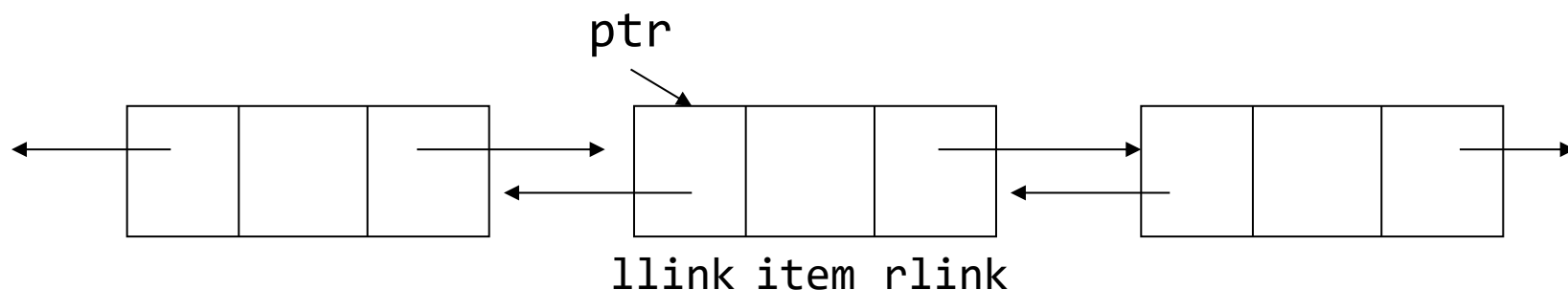
```
void delete(listPointer *first, listPointer trail,
listPointer x)
{ /* 从列表中删除 x , trail 是 前驱节点, *first 是列表
的前部*/
    if (trail)
        trail->link = x->link;
    else
        *first = (*first)->link;
    free(x);
}
```

双向链表

❖ 节点结构

```
typedef struct node* nodePointer;  
typedef struct node {  
    nodePointer llink;  
    element item;  
    nodePointer rlink;  
}
```

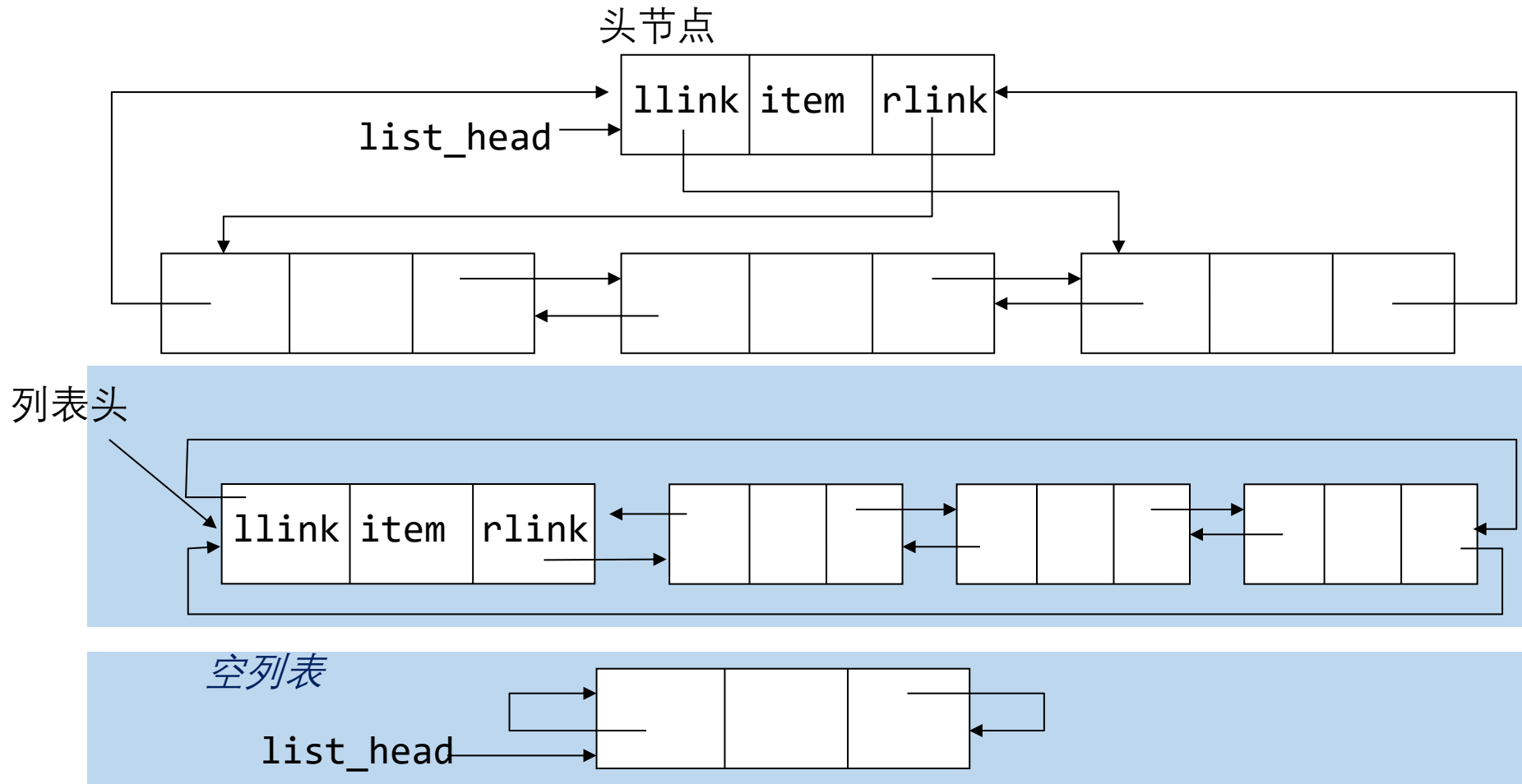
→ $ptr = ptr->llink->rlink = ptr->rlink->llink$



双向循环链表

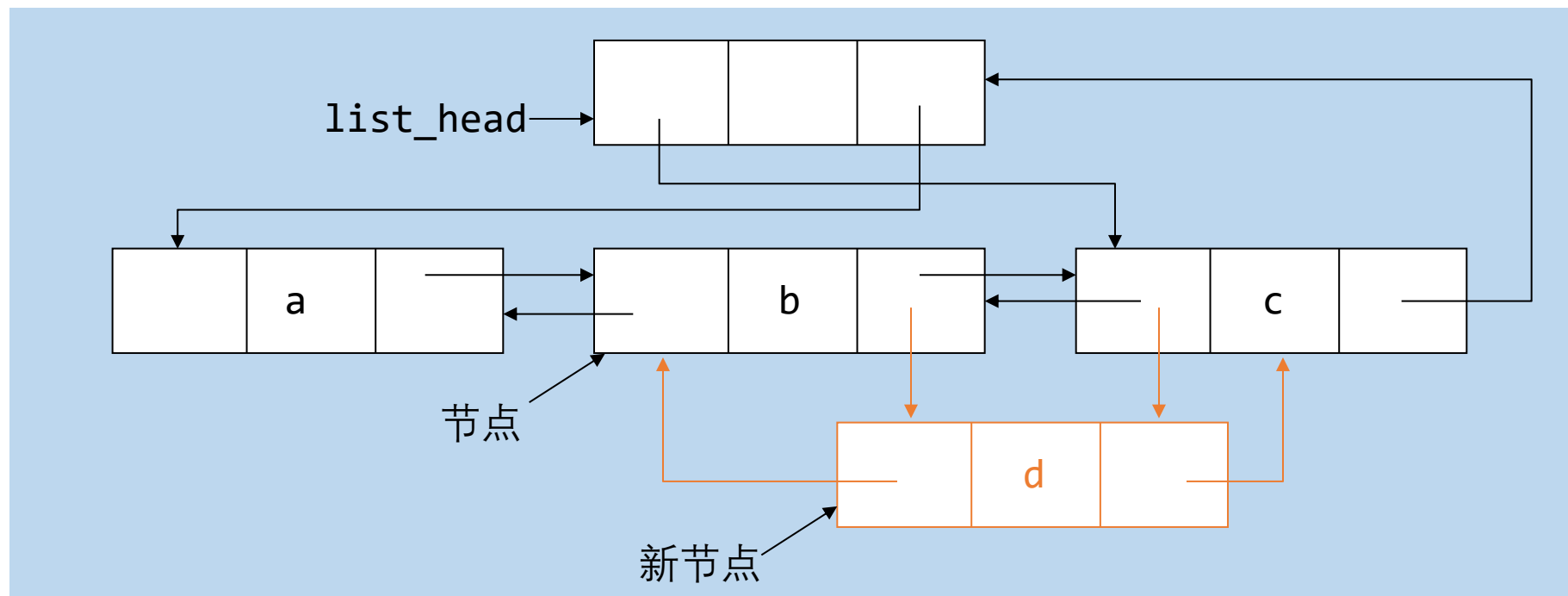
[Doubly Linked Circular Lists] (DLCL)

❖ 带有头节点的双向循环列表



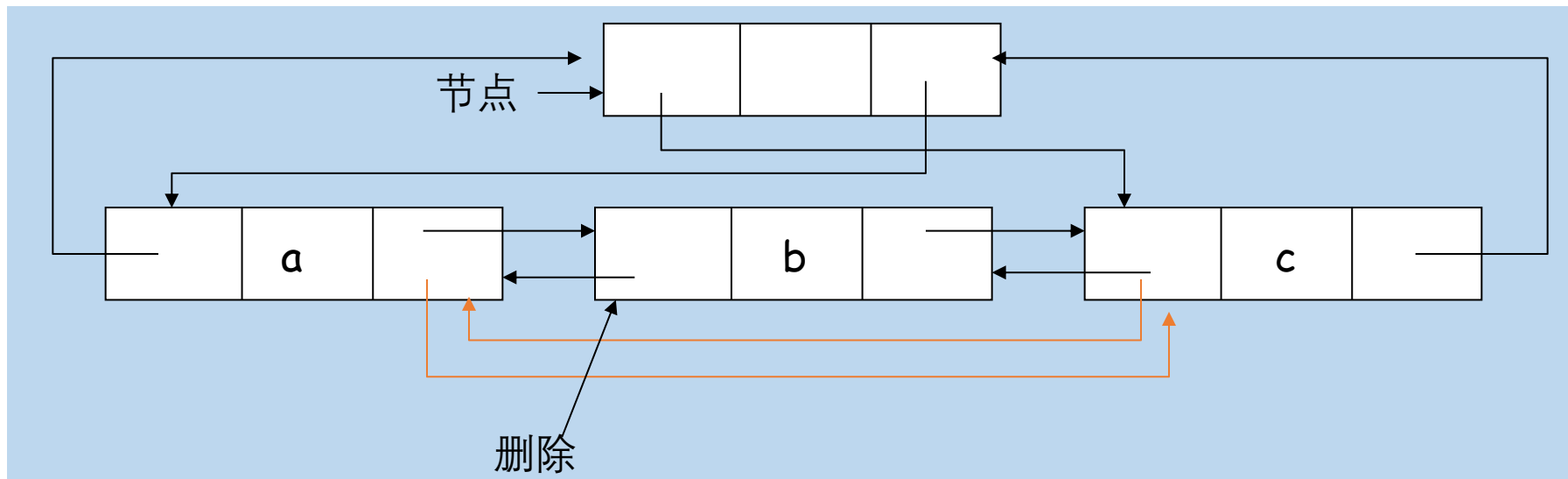
插入节点到 DLCL

```
void dinsert (nodePointer node, nodePointer newnode)
{ /* 在节点的右边插入新节点*/
  newnode->llink = node;
  newnode->rlink = node->rlink;
  node->rlink->llink = newnode;
  node->rlink = newnode; }
```



从 DLCL 删除节点

```
void ddelete (nodePointer node, nodePointer deleted)
{ /* 在DLCL中删除指针指向的节点*/
    if (node == deleted)
        printf("Deletion of head node not permitted.\n");
    else {
        deleted->llink->rlink = deleted->rlink;
        deleted->rlink->llink = deleted->llink;
        free(deleted);
    }
}
```



Next Topic

树 [Trees]

